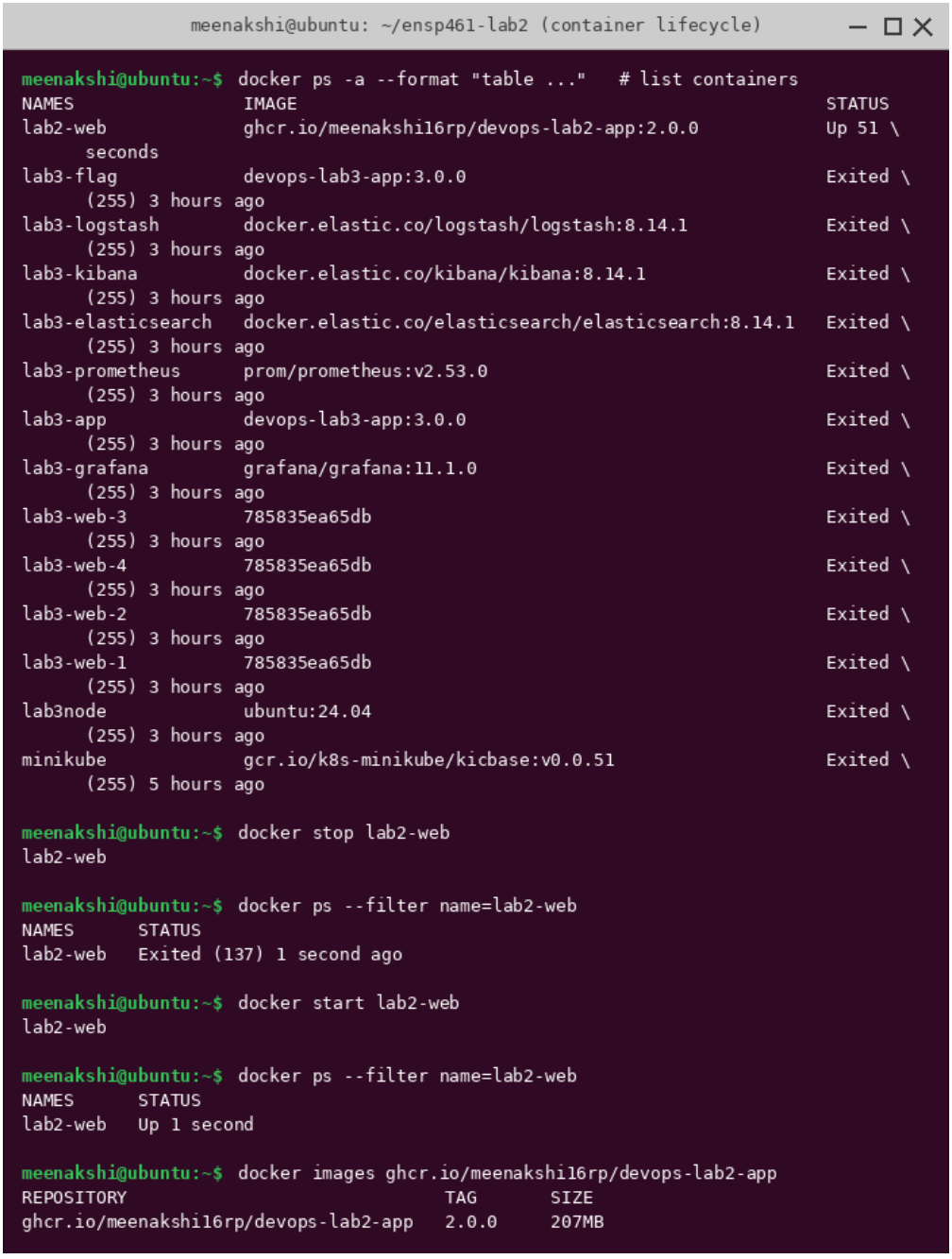
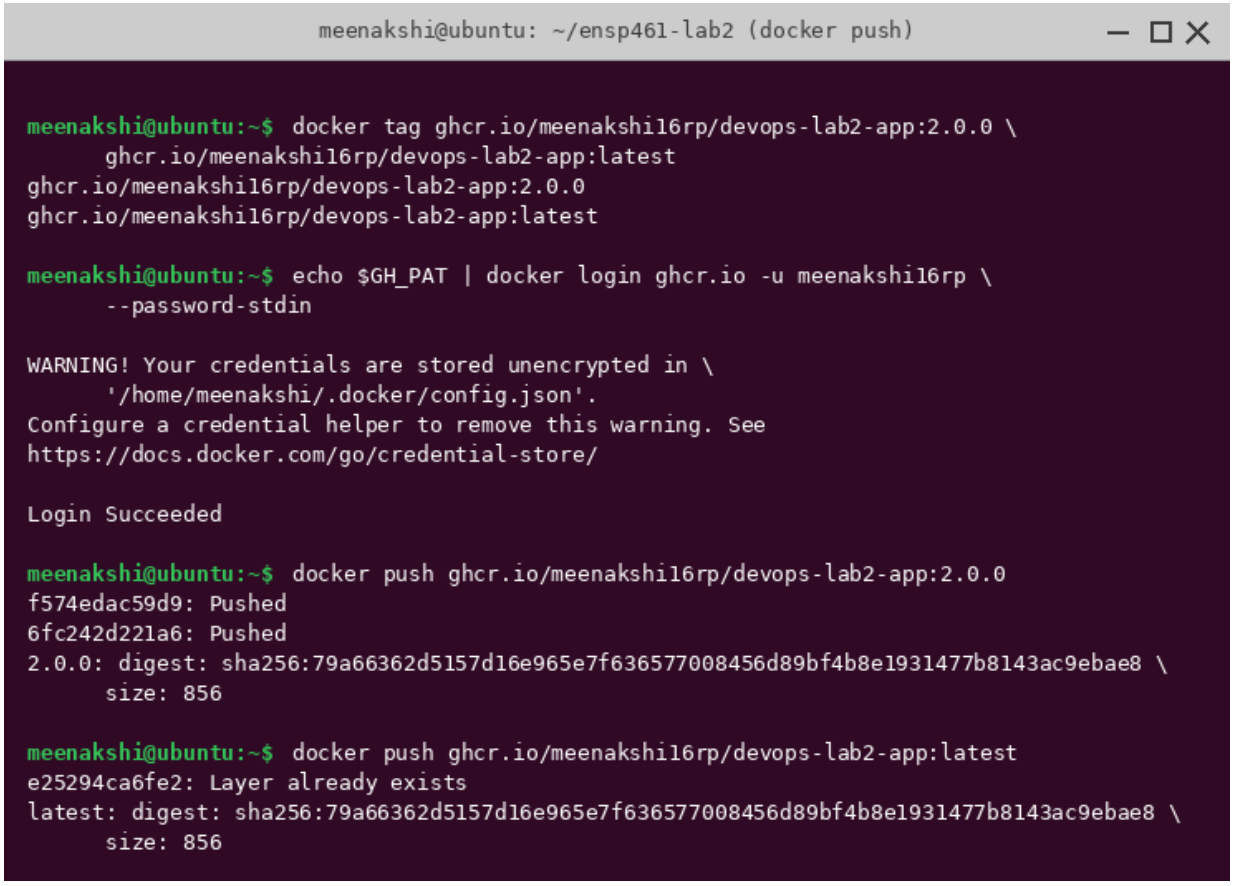
Question 2: Manage Docker containers and images: list, start, stop, remove containers, tag and push images to a registry, and use Docker Compose for multi-container setups.

Executed on Ubuntu 24.04 LTS (WSL2) by Meenakshi Rakesh. Repository: https://github.com/meenakshi16rp/Devops-lab-2

1. Container lifecycle (list, stop, start, tag)



2. Tag and push to a registry (ghcr.io)

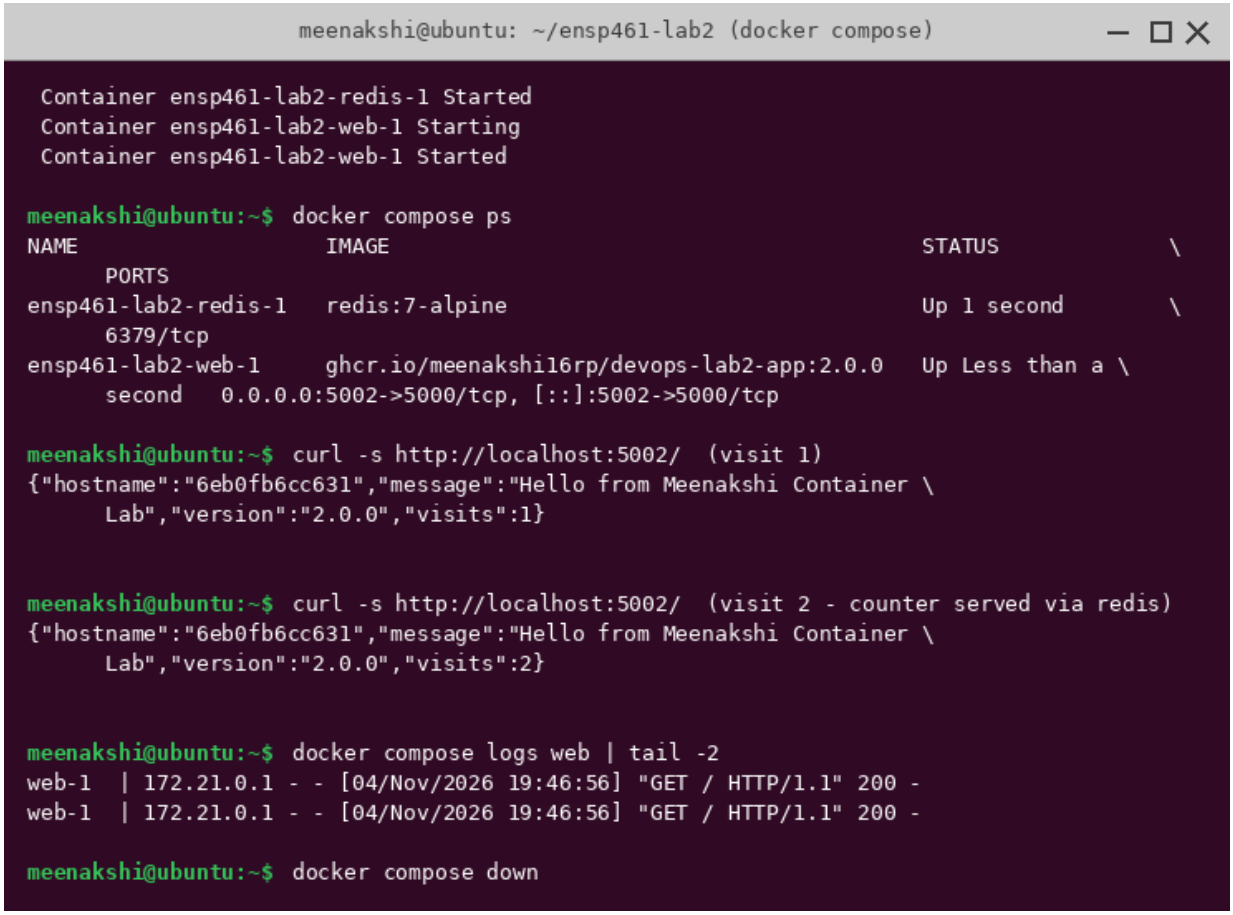


- The image is tagged ghcr.io/meenakshi16rp/devops-lab2-app:2.0.0 and :latest, then pushed with a GitHub PAT (docker login ghcr.io --password-stdin).
- Digest after push: sha256:79a66362... - the same manifest as the local image.

3. Docker Compose - multi-container setup (web + redis)

docker-compose.yml

```
services:
  web:
    build: .
    image: ghcr.io/meenakshi16rp/devops-lab2-app:2.0.0
    ports:
      - "5002:5000"
    environment:
      REDIS_HOST: redis
    depends_on:
      - redis
  redis:
    image: redis:7-alpine
```



- The web container counts visits in redis: two requests returned visits=1 and visits=2, proving cross-container communication inside the Compose network.
- docker compose down removes the containers and the Compose network together.

4. Observations

- docker stop sends SIGTERM (exit 137 after timeout) and docker start resumes the same container with its filesystem intact.
- Compose gives each service a DNS name (redis, web) on a private network - no manual docker network create needed.
- Registry pushes transfer only the missing layers: the :latest push reused the layers already uploaded for :2.0.0 (Layer already exists).